VERITAS 智鉴 · 音频鉴伪

音频伪造区间定位机制与折线图可视化可行性分析

说明当前算法如何判断"哪一段是伪造的"，以及在现有数据条件下为什么不适合用折线图呈现。

版本 v1 · 2026-10-31 · 适用：AI 音频伪造检测（4.1）

一、结论摘要

> 当前系统输出的"伪造区间"来自两种完全不同的机制：**短音频（≤30 秒）是模型自己用文字写出来的**，**长音频（>30 秒）是我们对滑窗分数做阈值合并算出来的。**两种机制的分辨率都在"秒到十秒"量级，且短音频路径经实测不可靠。
>
> 因此，把区间数据画成折线图会制造一种并不存在的"逐秒精度"。建议改用**时间轴色块**或**窗口柱状图**表达"哪一段可疑"；若必须用线，应使用阶梯线并明确标注为"窗口级评分"。

二、当前算法如何判别伪造区间

2.1 两条路径

系统的输入是任意长度的音频或视频（视频先抽取音轨）。推理引擎按**时长**选择不同的定位策略，核心参数为：

单段推理上限 `chunk_sec = 30` 秒

滑窗长度 `window_sec = 10` 秒

滑窗步长 `hop_sec = 5` 秒（相邻窗口 50% 重叠）

判定阈值 `0.5`（模型默认值，尚未按业务场景校准）

输入时长	处理方式	区间来源
≤ 30 秒	整段一次推理	**模型生成文本中的** `fake_region`，原样透传
> 30 秒	10 秒窗、5 秒步长滑窗，逐窗推理	**由系统对窗口分数做阈值合并**得到绝对时间

2.2 短音频路径：区间是"模型说出来的"

模型被要求输出一段结构化 JSON，其中 `fake_region` 是它逐 token 生成的内容，有两种形态：

```
{"real_or_fake": "fake", "fake_region": "all",  "spoof_method": "text to speech synthesis"}
{"real_or_fake": "fake", "fake_region": [[1.26, 1.64]], "spoof_method": "cut and paste"}
```

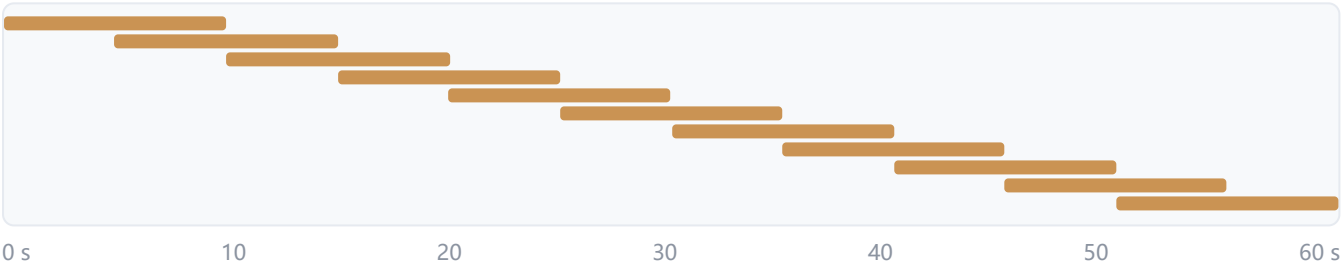
引擎只做解析，不做二次加工：

```
parsed = json.loads(result["raw"])
region = parsed.get("fake_region")   # "all" 或 [[start, end], ...]
```

> **实测局限**：在人工构造的局部篡改样本上，模型给出的区间**方向是相反的**——它指向的是真人那一段，而不是被替换的合成段。例如把 TTS 片段插到真人录音的 1.4–2.2 秒，模型返回 `[[0.45, 1.4]]`；把 TTS 放在开头（0–1.8 秒），模型返回 `[[1.8, 2.39]]`。因此该区间目前**不能作为证据级定位**使用。

2.3 长音频路径：滑窗打分后合并

模型只在数秒级片段上训练过，且 token 数随时长线性增长（实测约 35 token/秒），无法整段处理。因此长音频采用滑动窗口：



每个色条是一个 10 秒窗口；相邻窗口起点相差 5 秒（50% 重叠）。60 秒音频共 11 个窗口。

重叠 50% 的原因：若按 10 秒硬切，一段 2 秒的篡改恰好落在切点上时，两侧窗口都只看到半段，可能都判不出来；5 秒步长保证任意 ≤10 秒的篡改必定被某个窗口完整覆盖。

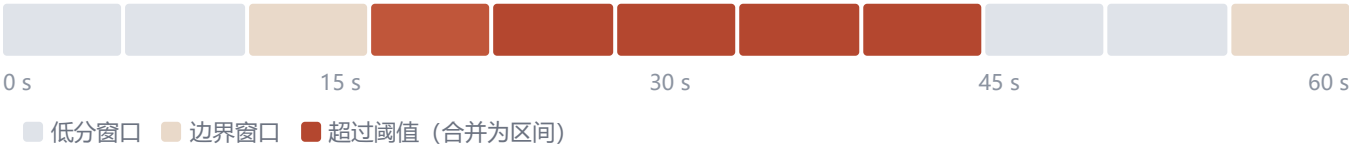
每个窗口独立推理，得到一个伪造分数；随后按阈值筛选并合并成绝对时间：

```
hot = [w for w in windows if w["pFake"] >= 0.5]   # 超过阈值的窗口

if not hot:                    region = None       # 无区间
elif len(hot) == len(windows): region = "all"      # 全段
else:                          region = 合并重叠区间(hot)
```

以一条 60 秒混合样本（真实篡改位于 20–40 秒）为例，11 个窗口的实测分数为：

窗口	0–10	5–15	10–20	15–25	20–30	25–35	30–40	35–45	40–50	45–55	50–60
pFake	0.001	0.003	0.029	0.984	0.997	1.000	0.999	0.999	0.001	0.003	0.029



合并结果：`[[15, 45]]`。真实篡改区间是 20–40 秒，误差约 ±5 秒，即**半个到一个窗宽**——这是 10 秒窗口的固有精度上限，不是实现问题。

2.4 一个曾经踩过的坑

早期版本把"分数最高的那个窗口"直接当作区间。问题在于 argmax 永远会选出一个——即使是**纯真人的 60 秒音频，也会被报出一段"可疑区间"**，属于凭空造出的定位。现版本改为"只合并超过阈值的窗口"，无窗口超阈值则返回 `null`。

2.5 精度与成本

指标	实测值
窗口分辨率	10 秒窗 / 5 秒步长，边界误差约 ±5–10 秒
60 秒音频窗口数	11 个
4 秒音频推理耗时	2–6 秒（GPU 空闲时）
60 秒音频推理耗时	23–38 秒（每窗一次完整推理）
共享 GPU 拥塞时	单条可退化到 44–58 秒

三、为什么不适合做成折线图

3.1 数据形态不支持：多数情况下只有一个点

折线图需要"一串随时间变化的数值"。但系统的现状是：

- **≤30 秒的音频**走整段推理，**不产生任何窗口**，只有 1 个总分。演示用的 4 秒样例就属于这一类——画出来是一个孤点，不是线。
- **>30 秒的音频**才有窗口数组，60 秒 11 个点、10 分钟约 119 个点。

也就是说，**越是短音频（演示与日常使用的主流场景），越没有曲线可言。**若要强行给 4 秒音频画线，必须切成 1 秒窗跑 7 次推理，把 2 秒的延迟抬到 20 秒以上。

3.2 分辨率撑不起"曲线"的外观

11 个点画成平滑曲线，视觉上会呈现"每时每刻都在测量"的效果，而实际采样间隔是 10 秒。曲线在两点之间插值出来的中间值（例如 0.63）在物理上并不存在——**没有任何一次推理产生过这个数。**

3.3 语义不成立：窗口之间是独立采样，不是连续分布

每个窗口的分数是**独立推理**得到的 token softmax 值，窗口之间没有时序连续性，也不构成一条概率密度曲线。折线图天然暗示"这是一条连续的置信度分布"，而我们的数据是"若干互不相关的离散判定"。

此外，该分数**尚未校准**——它是生成词表中 real / fake 两个 token 的 softmax，只保证序关系（越大越像伪造），不能解释为"有 87% 的概率是假的"。折线图会把这种误解放大。

3.4 可靠性不足以支撑精细表达

我们在人工构造的局部篡改样本上做过系统实验：模型能判出"这条是伪造的"（分数 0.98–1.00），但**给出的区间方向是反的**；长音频滑窗的边界误差约 ±10 秒。把这样的数据渲染成平滑曲线，等于把"我们知道得比较粗"包装成"我们测得很细"，一旦被追问"为什么这一段是 0.6"，无法给出可辩护的解释。

3.5 成本会随精度要求急剧上升

目标精度	10 分钟音频的推理次数	估算耗时
10 秒窗 / 5 秒步长（现状）	约 119 次	约 6–10 分钟
1 秒窗 / 0.5 秒步长	约 1199 次	约 60–100 分钟

按单窗约 3 秒、GPU 共享环境估算；且精度提升仅体现在采样密度，不改变判定的可靠性上限。

四、替代方案建议

4.1 方案 A：时间轴色块（推荐）

用一条横向色带表达"哪几段可疑"，颜色分档（如 <0.5 灰、0.5–0.9 浅红、≥0.9 深红），下方标注时间刻度。

> **优点**：视觉上一眼看出"哪里有问题"；不暗示连续概率；粗就是粗，语义诚实；前端实现成本低于折线图（若干色块即可）。
> **适用**：所有场景，尤其是需要在报告里说明"可疑片段"时。

4.2 方案 B：窗口柱状图

每个窗口一根柱，柱高为分数。柱状图天然表达"离散采样"，不会让人误以为中间有值。

缺点：窗口数少时（如 11 根）观感一般；需要处理重叠窗口的横轴对齐。

4.3 方案 C：阶梯线（仅在必须用"线"时）

若业务方坚持折线形态，建议：

- 使用**阶梯线**（step chart）而非平滑曲线，明确表示"窗口内取值恒定"；
- 仅对 >30 秒的输入展示，≤30 秒只显示单值与整段判定；
- 横轴刻度标注为**窗口中心**，标题写"窗口评分（10 秒窗 / 5 秒步长）"，避免使用"置信度曲线"；
- 在图上标注判定阈值线（0.5），让人看到判定依据。

五、结论

1. **折线图不适合当前数据。**核心原因是数据形态（多数输入只有 1 个点）、分辨率（10 秒级）、语义（独立采样而非连续分布）与可靠性（区间方向经实测不可靠）四方面都不支撑。

2. **"希望看到时间维度"的诉求是合理的**，建议用时间轴色块或窗口柱状图实现，二者都能表达"哪一段可疑"，且不会制造精度幻觉。
3. **若后续要真正提升定位精度**，路径是：缩小窗口（成本线性上升）、使用单词级标注数据（如 PartialEdit）微调、或采用少样本上下文示例（ICL），而不是改变可视化形式。

本文数据来自 2026-10 在本项目推理服务上的实测：4 条演示样例、60 秒真人/伪造/混合样本、以及 11 组人工构造的局部篡改样本。区间定位的实现见推理服务 `engine.py` 中 `Engine.infer_file()` 的窗口合并逻辑。